5.7 Exercises

Reinforcement

R-5.1 Describe a recursive algorithm for finding the maximum element in an array, A, of n elements. What is your running time and space usage?

R-5.2 Explain how to modify the recursive binary search algorithm so that it returns the index of the target in the sequence or -1 (if the target is not found).

R-5.3 Draw the recursion trace for the computation of $power(2,5)$, using the traditional algorithm implemented in Code Fragment 5.8.

R-5.4 Draw the recursion trace for the computation of $power(2,18)$, using the repeated squaring algorithm, as implemented in Code Fragment 5.9.

R-5.5 Draw the recursion trace for the execution of reverseArray(data, 0, 4), from Code Fragment 5.7, on array data $=$ 4, 3, 6, 2, 6.

R-5.6 Draw the recursion trace for the execution of method PuzzleSolve$(3,S,U)$, from Code Fragment 5.11, where S is empty and $U = \{a,b,c,d\}$.

R-5.7 Describe a recursive algorithm for computing the n^{th} ***Harmonic number***, defined as $H_n = \sum_{k=1}^{n} 1/k$.

R-5.8 Describe a recursive algorithm for converting a string of digits into the integer it represents. For example, `'13531'` represents the integer $13,531$.

R-5.9 Develop a nonrecursive implementation of the version of the power method from Code Fragment 5.9 that uses repeated squaring.

R-5.10 Describe a way to use recursion to compute the sum of all the elements in an $n \times n$ (two-dimensional) array of integers.

Creativity

C-5.11 Describe a recursive algorithm to compute the integer part of the base-two logarithm of n using only addition and integer division.

C-5.12 Describe an efficient recursive algorithm for solving the element uniqueness problem, which runs in time that is at most $O(n^2)$ in the worst case without using sorting.

C-5.13 Give a recursive algorithm to compute the product of two positive integers, m and n, using only addition and subtraction.

C-5.14 In Section 5.2 we prove by induction that the number of *lines* printed by a call to drawInterval(c) is $2^c - 1$. Another interesting question is how many *dashes* are printed during that process. Prove by induction that the number of dashes printed by drawInterval(c) is $2^{c+1} - c - 2$.

C-5.15 Write a recursive method that will output all the subsets of a set of n elements (without repeating any subsets).

C-5.16 In the ***Towers of Hanoi*** puzzle, we are given a platform with three pegs, a, b, and c, sticking out of it. On peg a is a stack of n disks, each larger than the next, so that the smallest is on the top and the largest is on the bottom. The puzzle is to move all the disks from peg a to peg c, moving one disk at a time, so that we never place a larger disk on top of a smaller one. See Figure 5.15 for an example of the case $n = 4$. Describe a recursive algorithm for solving the Towers of Hanoi puzzle for arbitrary n. (Hint: Consider first the subproblem of moving all but the n^{th} disk from peg a to another peg using the third as "temporary storage.")

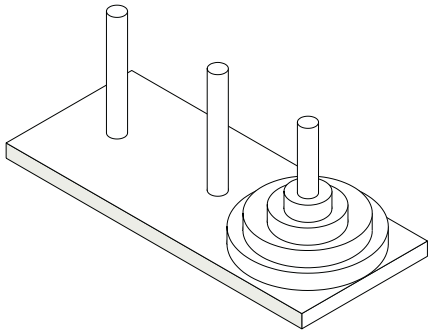

Figure 5.15: An illustration of the Towers of Hanoi puzzle.

C-5.17 Write a short recursive Java method that takes a character string s and outputs its reverse. For example, the reverse of `'pots&pans'` would be `'snap&stop'`.

C-5.18 Write a short recursive Java method that determines if a string s is a palindrome, that is, it is equal to its reverse. Examples of palindromes include `'racecar'` and `'gohangasalamiimalasagnahog'`.

C-5.19 Use recursion to write a Java method for determining if a string s has more vowels than consonants.

C-5.20 Write a short recursive Java method that rearranges an array of integer values so that all the even values appear before all the odd values.

C-5.21 Given an unsorted array, A, of integers and an integer k, describe a recursive algorithm for rearranging the elements in A so that all elements less than or equal to k come before any elements larger than k. What is the running time of your algorithm on an array of n values?

C-5.22 Suppose you are given an array, A, containing n distinct integers that are listed in increasing order. Given a number k, describe a recursive algorithm to find two integers in A that sum to k, if such a pair exists. What is the running time of your algorithm?

C-5.23 Describe a recursive algorithm that will check if an array A of integers contains an integer $A[i]$ that is the sum of two integers that appear earlier in A, that is, such that $A[i] = A[j] + A[k]$ for $j, k < i$.

C-5.24 Isabel has an interesting way of summing up the values in an array A of n integers, where n is a power of two. She creates an array B of half the size of A and sets $B[i] = A[2i] + A[2i+1]$, for $i = 0, 1, \ldots, (n/2) - 1$. If B has size 1, then she outputs $B[0]$. Otherwise, she replaces A with B, and repeats the process. What is the running time of her algorithm?

C-5.25 Describe a fast recursive algorithm for reversing a singly linked list L, so that the ordering of the nodes becomes opposite of what it was before.

C-5.26 Give a recursive definition of a singly linked list class that does not use any Node class.

Projects

P-5.27 Implement a recursive method with calling signature `find(path, filename)` that reports all entries of the file system rooted at the given path having the given file name.

P-5.28 Write a program for solving summation puzzles by enumerating and testing all possible configurations. Using your program, solve the three puzzles given in Section 5.3.3.

P-5.29 Provide a nonrecursive implementation of the `drawInterval` method for the English ruler project of Section 5.1.2. There should be precisely $2^c - 1$ lines of output if c represents the length of the center tick. If incrementing a counter from 0 to $2^c - 2$, the number of dashes for each tick line should be exactly one more than the number of consecutive 1's at the end of the binary representation of the counter.

P-5.30 Write a program that can solve instances of the Tower of Hanoi problem (from Exercise C-5.16).

Chapter Notes

The use of recursion in programs belongs to the folklore of computer science (for example, see the article of Dijkstra [31]). It is also at the heart of functional programming languages (for example, see the book by Abelson, Sussman, and Sussman [1]). Interestingly, binary search was first published in 1946, but was not published in a fully correct form until 1962. For further discussions on lessons learned, see papers by Bentley [13] and Lesuisse [64].